Итоговая работа

Инфраструктура мониторинга echo-API

Bash-скрипты, анализ сетевого трафика, Docker Compose и CI/CD на GitHub Actions

Уральский федеральный университет (УрФУ), РТФ

Дисциплина: «Программная инженерия»

Студент: Зиганшин Дмитрий Алексеевич

Группа: РИМ-160980к

Преподаватель: Меркушов Михаил

Екатеринбург, 2026

Материалы работы

Материал	Ссылка
Репозиторий	https://github.com/onezsie/infra-monitoring-project/tree/feature/monitoring
Pull Request feature/monitoring → main	https://github.com/onezsie/infra-monitoring-project/pull/1
Дамп трафика (traffic.pcapng)	https://github.com/onezsie/infra-monitoring-project/blob/feature/monitoring/report/traffic.pcapng

1. Репозиторий и Git

Проект размещён в публичном репозитории onezsie/infra-monitoring-project. Ветка `main` защищена: изменения попадают в неё только через Pull Request. Работа велась в ветке `feature/monitoring`. Файл `.gitignore` исключает логи, временные файлы и артефакты сборки. Изменения оформлены в Pull Request #1 в ветку `main` с описанием.

2. Анализ сетевого трафика

Трафик записан в Wireshark на интерфейсе `utun6`: на рабочей машине весь трафик проходит через туннельный интерфейс, адрес клиента в нём — `198.18.0.1`. Через `curl` выполнены HTTP-запросы к httpbin.org и HTTPS-запросы к httpbin.org и api.github.com. Дамп сохранён в файл report/traffic.pcapng.

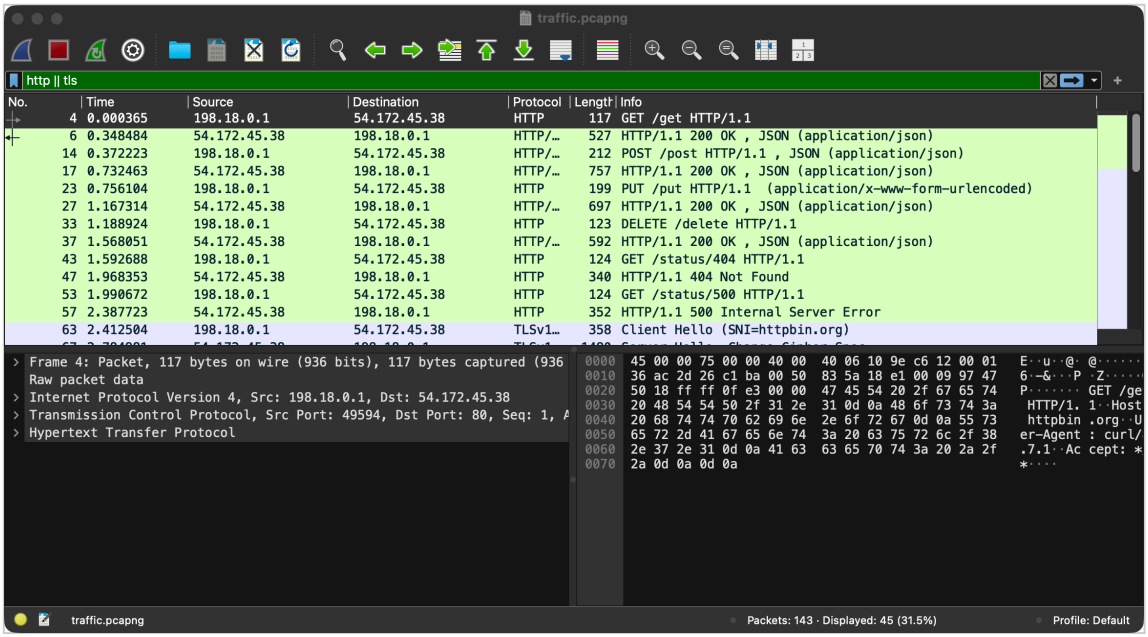


Рисунок 1 — Wireshark: трафик с фильтром http || tls

IP-адреса, участвовавшие в обмене

IP-адрес	Узел
`198.18.0.1`	клиент (рабочая машина)
`54.172.45.38`	httpbin.org
`140.82.121.5`	api.github.com

HTTP-методы и коды ответов

Метод	URI	Код ответа
GET	`/get`	200 OK
POST	`/post`	200 OK
PUT	`/put`	200 OK
DELETE	`/delete`	200 OK
GET	`/status/404`	404 Not Found
GET	`/status/500`	500 Internal Server Error

Использованы методы GET, POST, PUT и DELETE. Сервер вернул коды 200, 404 и 500. Содержимое HTTPS-соединений зашифровано TLS, поэтому методы и коды ответов в них в дампе не видны.

3. Bash-скрипты

3.1. check_api.sh

Скрипт принимает URL аргументом, делает запрос через `curl` и проверяет код ответа.

```
dmitrij@MacBook-Air-Dmitrij infra-monitoring-project % ./check_api.sh https://httpbin.org/status/200; echo $?
API доступен
0
dmitrij@MacBook-Air-Dmitrij infra-monitoring-project % ./check_api.sh not-a-url; echo $?
API недоступен, код ответа: 000
1
dmitrij@MacBook-Air-Dmitrij infra-monitoring-project %
```

Рисунок 2 — Работа check_api.sh с валидным и невалидным URL

Для валидного URL `https://httpbin.org/status/200` скрипт вывел «API доступен» и вернул 0. Для невалидного URL `not-a-url` — «API недоступен, код ответа: 000» и вернул 1.

3.2. analyze_logs.sh

Скрипт генерирует `access.log` на 300 строк и с помощью `grep`, `awk`, `sort`, `uniq` и регулярного выражения для IP-адресов выводит статистику по нему.

```
dmitrij@MacBook-Air-Dmitrij infra-monitoring-project % ./analyze_logs.sh
Общее количество запросов: 300
Топ-3 IP-адреса:
  192.168.1.10 - 120
  10.0.0.5 - 73
  192.168.1.15 - 40
Запросов с кодами 4xx: 89
Запросов с кодами 5xx: 79
dmitrij@MacBook-Air-Dmitrij infra-monitoring-project %
```

Рисунок 3 — Вывод analyze_logs.sh

Всего 300 запросов. Топ-3 IP-адреса: 192.168.1.10 (120), 10.0.0.5 (73), 192.168.1.15 (40). Запросов с кодами 4xx — 89, с кодами 5xx — 79.

4. Docker Compose

Сервис `api` (`hashicorp/http-echo`) отвечает текстом «Monitoring Service Active» на порту 8080. Сервис `proxy` (`nginx:alpine`) проксирует запросы с порта 80 на `api:8080`. Контейнеры связаны внутренней сетью `monitoring-net`.

```
dmitrij@MacBook-Air-Dmitrij infra-monitoring-project % docker-compose ps
NAME                                IMAGE                COMMAND                  SERVICE   CREATED              STATUS              PORTS
infra-monitoring-project-api-1      hashicorp/http-echo  "/http-echo -listen=…"   api       About a minute ago   Up About a minute   5678/tcp
infra-monitoring-project-proxy-1    nginx:alpine         "/docker-entrypoint.…"   proxy     About a minute ago   Up About a minute   0.0.0.0:80->80/tcp, [::]:80->80/tcp
dmitrij@MacBook-Air-Dmitrij infra-monitoring-project % curl http://localhost:80
Monitoring Service Active
dmitrij@MacBook-Air-Dmitrij infra-monitoring-project %
```

Рисунок 4 — Работающие контейнеры (docker-compose ps)

После `docker-compose up -d` оба контейнера работают, `curl http://localhost:80` возвращает «Monitoring Service Active».

5. CI/CD на GitHub Actions

При создании Pull Request пайплайн проверяет синтаксис скриптов (`bash -n check_api.sh`, `bash -n analyze_logs.sh`) и запускает `check_api.sh https://httpbin.org/status/200` с ожиданием кода возврата 0.

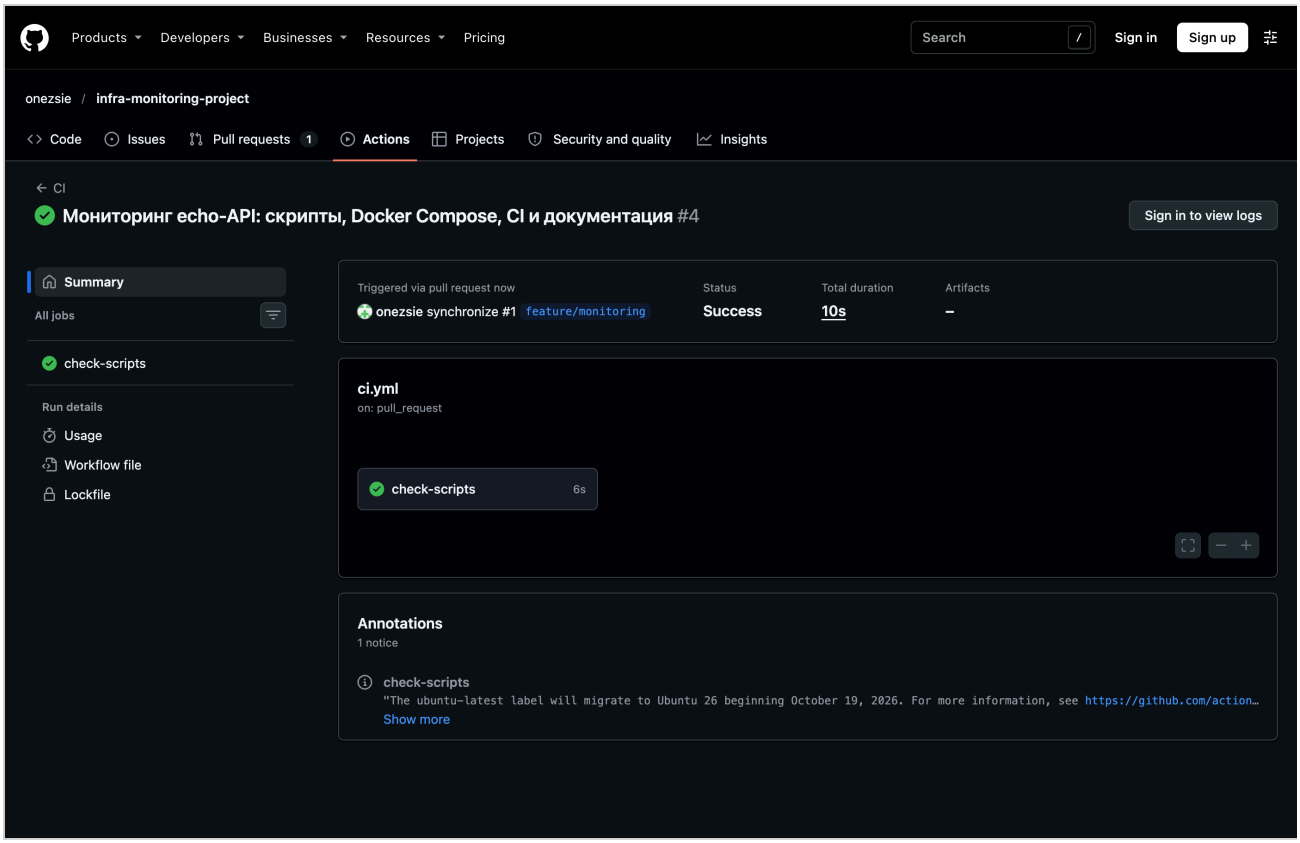


Рисунок 5 — Успешный прогон CI/CD-пайплайна на Pull Request

6. SSH и SCP

Процесс безопасной передачи проекта на удалённый сервер (`ssh-keygen`, `ssh-copy-id`, передача архива через `scp`) и команды для деплоя описаны в README.md.